

Task 'Introduction and Recap of Object-Oriented Programming (OOP)' Unit 1

Task 1: Basic Class Hierarchy (Inheritance)

class Vehicle:

```
def __init__(self, brand, fuel_type):
```

```
    # Initialize attributes
```

```
    self.brand = brand
```

```
    self.fuel_type = fuel_type
```

class Car(Vehicle):

```
def __init__(self, brand, fuel_type, num_doors):
```

```
    # Call parent constructor
```

```
    super().__init__(brand, fuel_type)
```

```
    # Initialize additional attribute
```

```
    self.num_doors = num_doors
```

Test case

```
car1 = Car("Toyota", "Petrol", 4)
```

```
print("Task 1 Output:")
```

```
print(car1.brand, car1.fuel_type, car1.num_doors)
```

```
print()
```

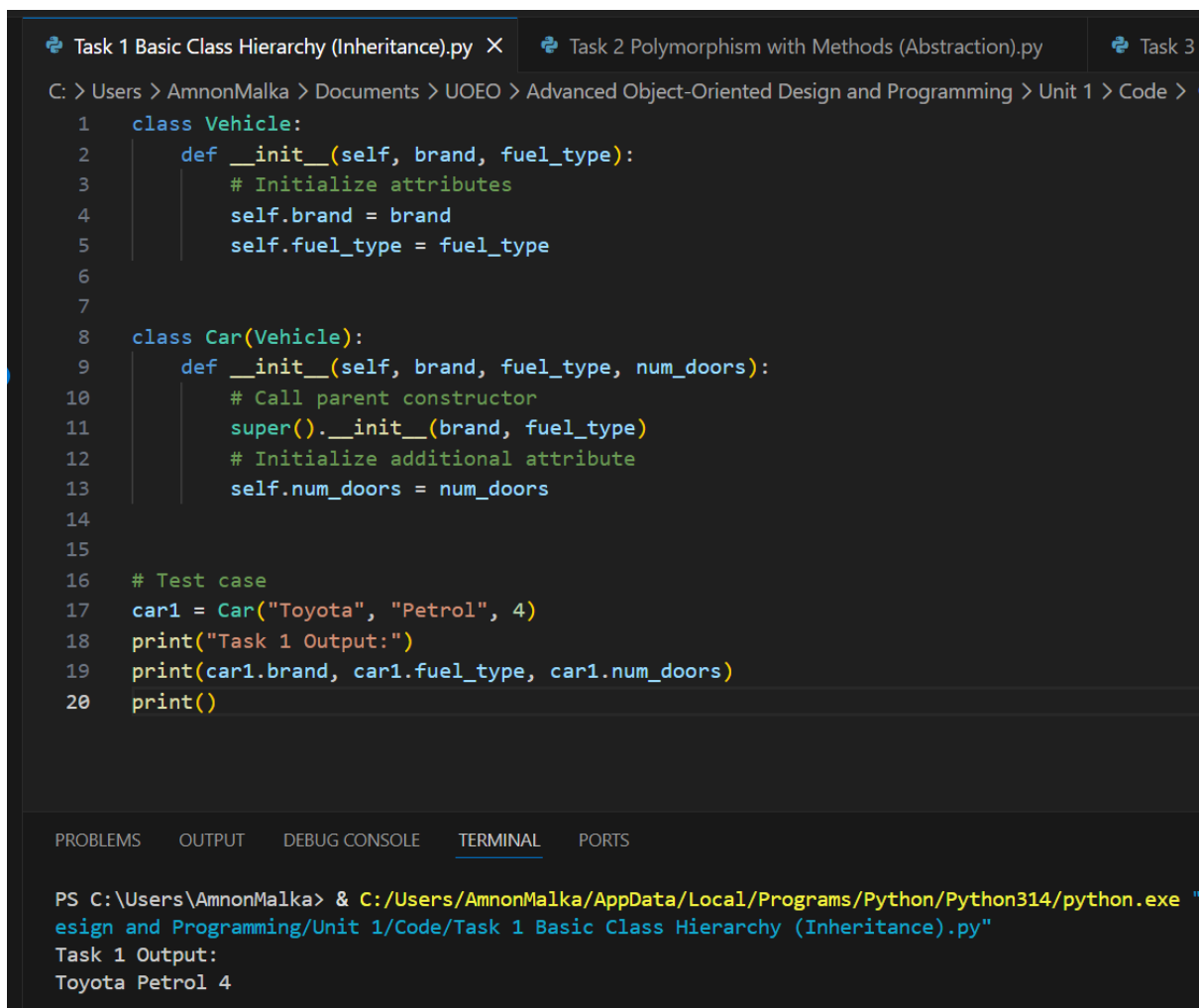
Result:

Task 'Introduction and Recap of Object-Oriented Programming (OOP)' Unit 1

This task demonstrates inheritance and reuse of parent class attributes which is one of the core concepts of object-oriented programming to support hierarchical relationships and code reuse.

(Lott, 2024; Downey, 2024).

Figure 1: Basic Class Hierarchy (Inheritance) Test Case Result



```
Task 1 Basic Class Hierarchy (Inheritance).py X Task 2 Polymorphism with Methods (Abstraction).py Task 3
C: > Users > AmnonMalka > Documents > UOEO > Advanced Object-Oriented Design and Programming > Unit 1 > Code >
1 class Vehicle:
2     def __init__(self, brand, fuel_type):
3         # Initialize attributes
4         self.brand = brand
5         self.fuel_type = fuel_type
6
7
8 class Car(Vehicle):
9     def __init__(self, brand, fuel_type, num_doors):
10        # Call parent constructor
11        super().__init__(brand, fuel_type)
12        # Initialize additional attribute
13        self.num_doors = num_doors
14
15
16 # Test case
17 car1 = Car("Toyota", "Petrol", 4)
18 print("Task 1 Output:")
19 print(car1.brand, car1.fuel_type, car1.num_doors)
20 print()

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\AmnonMalka> & C:/Users/AmnonMalka/AppData/Local/Programs/Python/Python314/python.exe "
esign and Programming/Unit 1/Code/Task 1 Basic Class Hierarchy (Inheritance).py"
Task 1 Output:
Toyota Petrol 4
```

Task 2: Polymorphism with Methods (Abstraction)

```
from abc import ABC, abstractmethod
```

```
import math
```

Task 'Introduction and Recap of Object-Oriented Programming (OOP)' Unit 1

```
class Shape(ABC):

    @abstractmethod

    def area(self):

        pass

class Circle(Shape):

    def __init__(self, radius):

        self.radius = radius

    def area(self):

        return math.pi * (self.radius ** 2)

class Rectangle(Shape):

    def __init__(self, length, width):

        self.length = length

        self.width = width

    def area(self):

        return self.length * self.width
```

Test case

```
shapes = [Circle(5), Rectangle(4, 6)]
```

Task 'Introduction and Recap of Object-Oriented Programming (OOP)' Unit 1

```
print("Task 2 Output:")
```

```
for shape in shapes:
```

```
    print(shape.area())
```

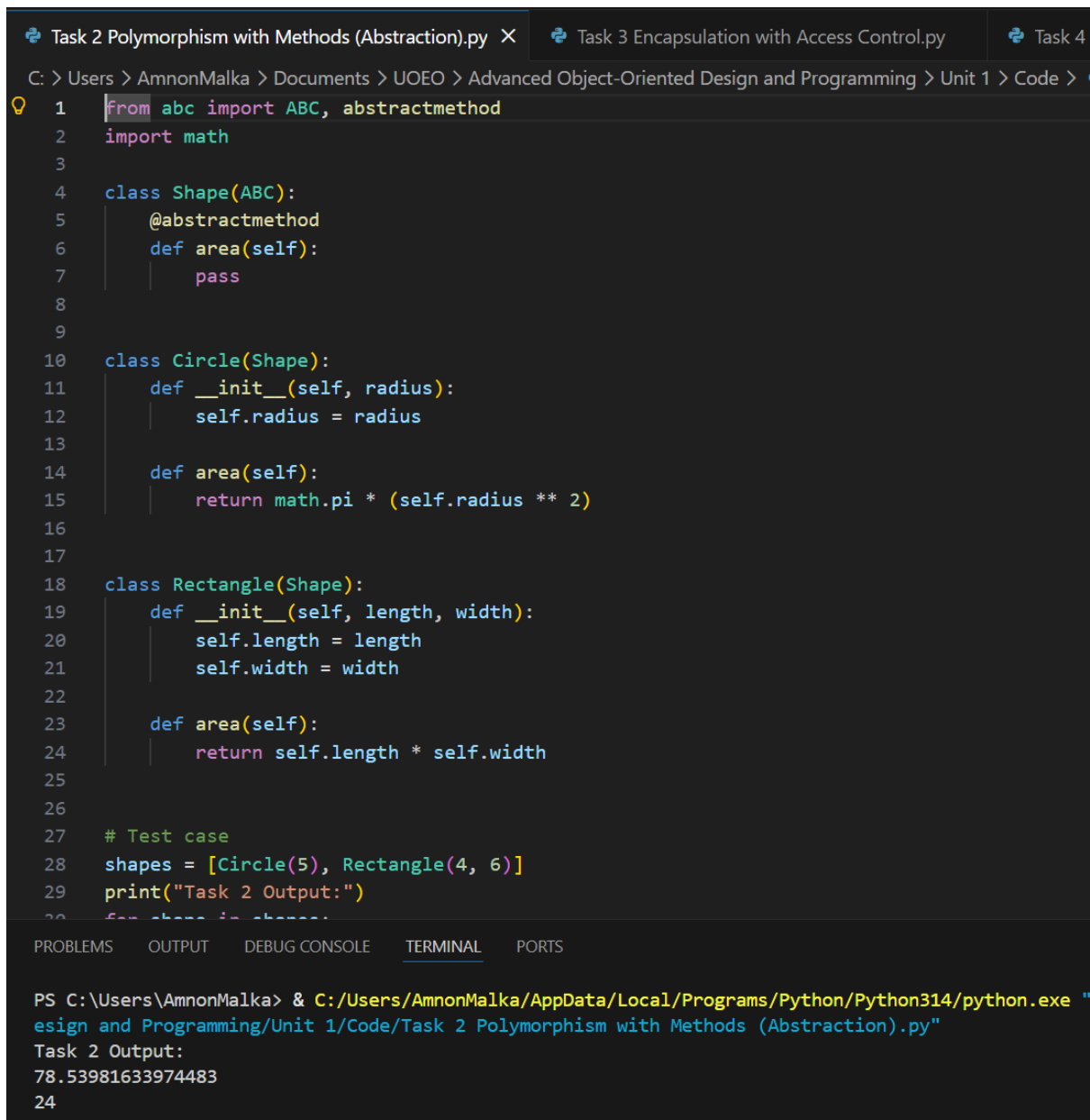
```
print()
```

Result:

The results had demonstrated polymorphism in action that adapted to condition using same method name, different behavior and that supported cleaner code employing abstraction in action.

Figure 2: Polymorphism with Methods (Abstraction) Test Case Result

Task 'Introduction and Recap of Object-Oriented Programming (OOP)' Unit 1



```
1 from abc import ABC, abstractmethod
2 import math
3
4 class Shape(ABC):
5     @abstractmethod
6     def area(self):
7         pass
8
9
10 class Circle(Shape):
11     def __init__(self, radius):
12         self.radius = radius
13
14     def area(self):
15         return math.pi * (self.radius ** 2)
16
17
18 class Rectangle(Shape):
19     def __init__(self, length, width):
20         self.length = length
21         self.width = width
22
23     def area(self):
24         return self.length * self.width
25
26
27 # Test case
28 shapes = [Circle(5), Rectangle(4, 6)]
29 print("Task 2 Output:")
30 for shape in shapes:
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\AmnonMalka> & C:/Users/AmnonMalka/AppData/Local/Programs/Python/Python314/python.exe "
esign and Programming/Unit 1/Code/Task 2 Polymorphism with Methods (Abstraction).py"
Task 2 Output:
78.53981633974483
24
```

Task 3: Encapsulation with Access Control

class BankAccount:

```
def __init__(self, initial_balance=0):
```

```
    # Private attribute
```

```
    self.__balance = initial_balance
```

Task 'Introduction and Recap of Object-Oriented Programming (OOP)' Unit 1

```
def deposit(self, amount):  
  
    if amount > 0:  
  
        self.__balance += amount  
  
def withdraw(self, amount):  
  
    if amount <= self.__balance:  
  
        self.__balance -= amount  
  
    else:  
  
        print("Insufficient funds")  
  
def get_balance(self):  
  
    return self.__balance
```

Test case

```
acc = BankAccount(100)  
  
acc.deposit(50)  
  
acc.withdraw(30)  
  
print("Task 3 Output:")  
  
print(acc.get_balance())  
  
print()
```

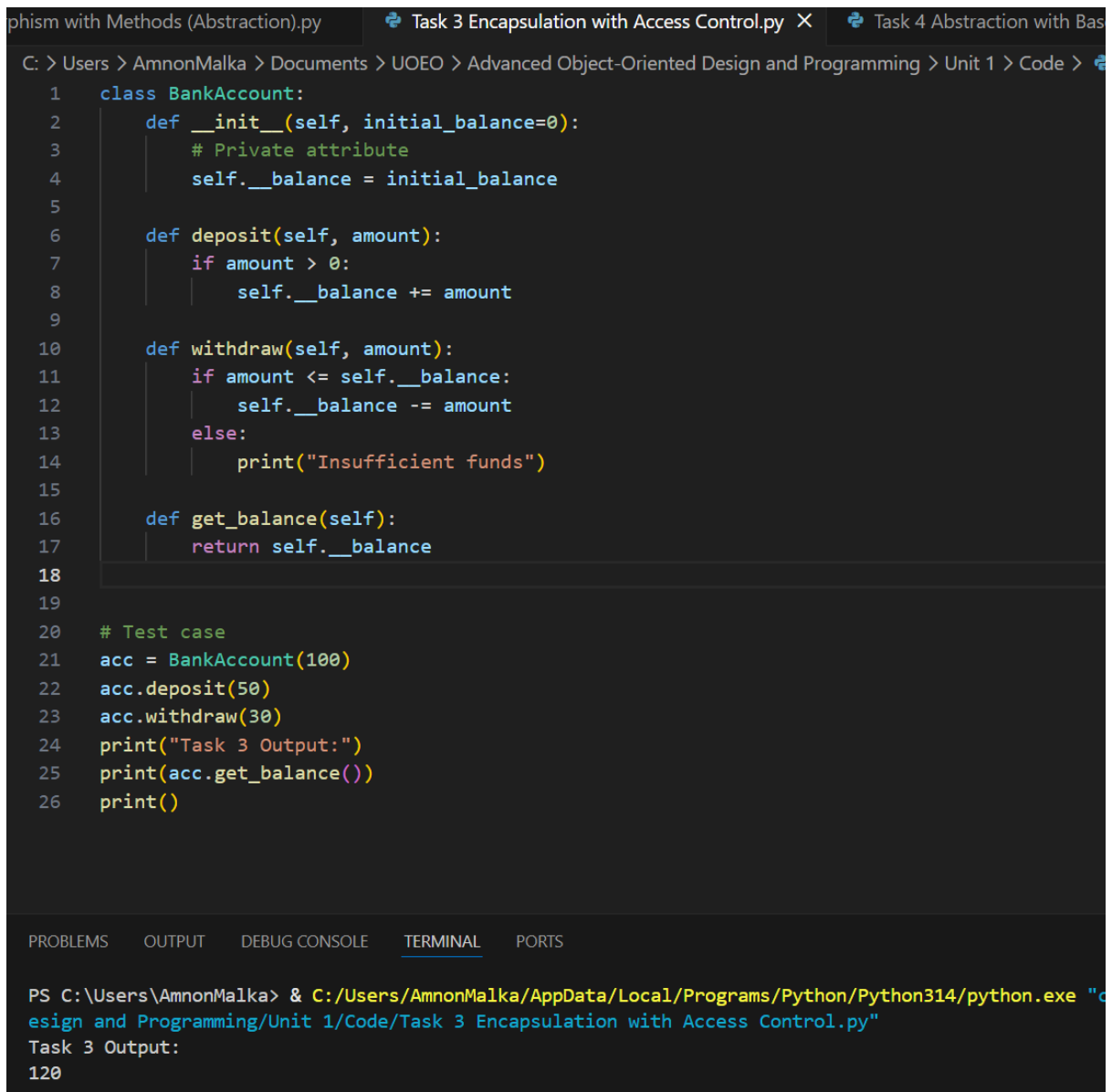
Result:

Task 'Introduction and Recap of Object-Oriented Programming (OOP)' Unit 1

Result had demonstrated encapsulation in action with the use of private attributes (`__balance`) and ensuring legitimate and controlled access to data hence protecting internal object state as intended.

(Lott, 2024).

Figure 3: Encapsulation with Access Control Test Case Result



```
phism with Methods (Abstraction).py Task 3 Encapsulation with Access Control.py X Task 4 Abstraction with Bas
C: > Users > AmnonMalka > Documents > UOEO > Advanced Object-Oriented Design and Programming > Unit 1 > Code >
1 class BankAccount:
2     def __init__(self, initial_balance=0):
3         # Private attribute
4         self.__balance = initial_balance
5
6     def deposit(self, amount):
7         if amount > 0:
8             self.__balance += amount
9
10    def withdraw(self, amount):
11        if amount <= self.__balance:
12            self.__balance -= amount
13        else:
14            print("Insufficient funds")
15
16    def get_balance(self):
17        return self.__balance
18
19
20 # Test case
21 acc = BankAccount(100)
22 acc.deposit(50)
23 acc.withdraw(30)
24 print("Task 3 Output:")
25 print(acc.get_balance())
26 print()

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\AmnonMalka> & C:/Users/AmnonMalka/AppData/Local/Programs/Python/Python314/python.exe "c
esign and Programming/Unit 1/Code/Task 3 Encapsulation with Access Control.py"
Task 3 Output:
120

```

Task 4: Abstraction with Base Class

Task 'Introduction and Recap of Object-Oriented Programming (OOP)' Unit 1

```
from abc import ABC, abstractmethod
```

```
class Animal(ABC):
```

```
    @abstractmethod
```

```
    def make_sound(self):
```

```
        pass
```

```
class Dog(Animal):
```

```
    def make_sound(self):
```

```
        return "Woof!"
```

```
class Cat(Animal):
```

```
    def make_sound(self):
```

```
        return "Meow!"
```

```
# Test case
```

```
animals = [Dog(), Cat()]
```

```
print("Task 4 Output:")
```

```
for animal in animals:
```

```
    print(animal.make_sound())
```

```
print()
```

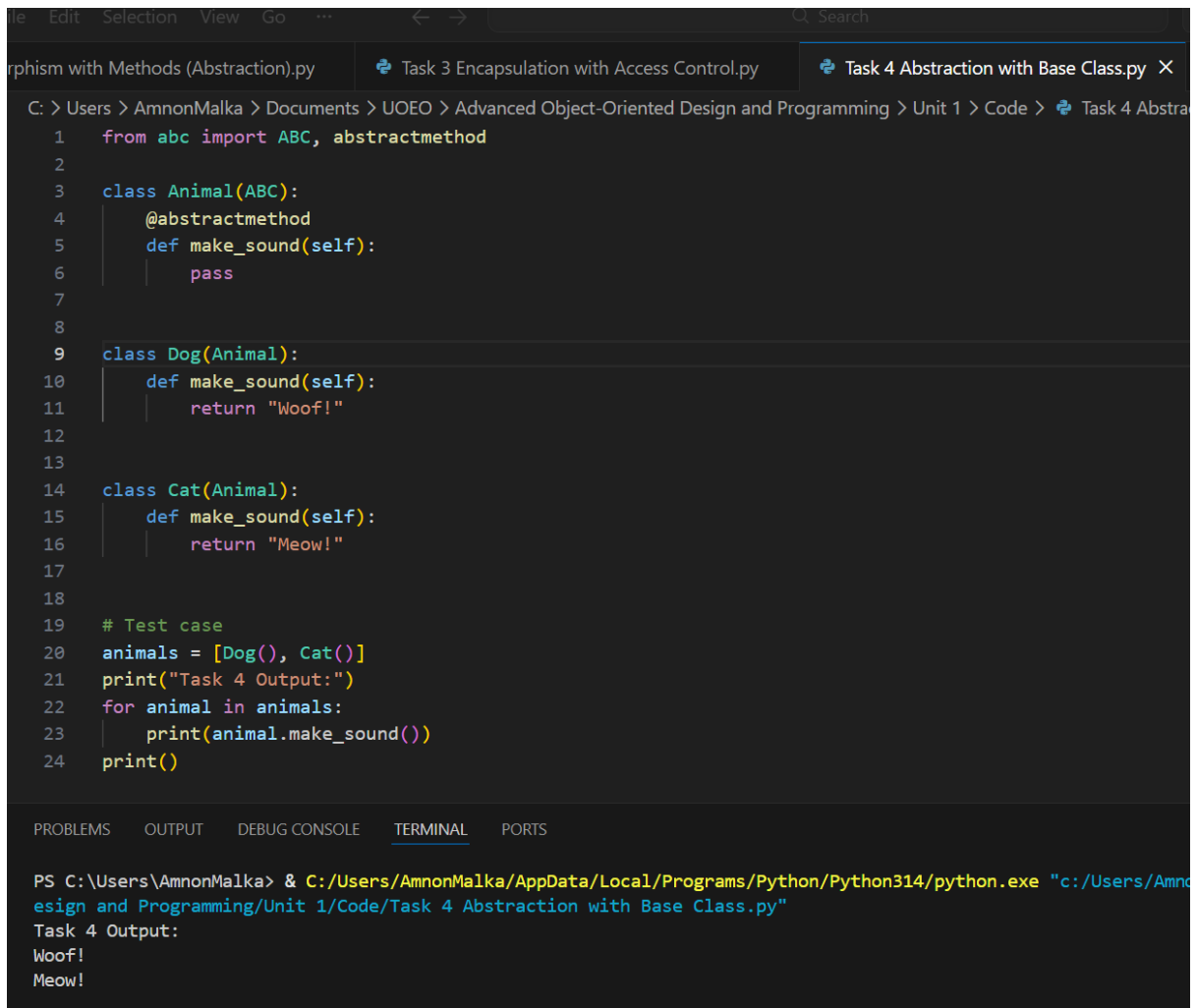

Task 'Introduction and Recap of Object-Oriented Programming (OOP)' Unit 1

Result:

Result had demonstrated abstraction in action with uniform interface design using abstract base classes which enforce method implementation in derived classes.

(Lott, 2024).

Figure 4: Abstraction with Base Class Test Case Result



```
File Edit Selection View Go ... Search
morphism with Methods (Abstraction).py Task 3 Encapsulation with Access Control.py Task 4 Abstraction with Base Class.py X
C: > Users > AmnonMalka > Documents > UOEO > Advanced Object-Oriented Design and Programming > Unit 1 > Code > Task 4 Abstra
1  from abc import ABC, abstractmethod
2
3  class Animal(ABC):
4      @abstractmethod
5      def make_sound(self):
6          pass
7
8
9  class Dog(Animal):
10     def make_sound(self):
11         return "Woof!"
12
13
14  class Cat(Animal):
15     def make_sound(self):
16         return "Meow!"
17
18
19  # Test case
20  animals = [Dog(), Cat()]
21  print("Task 4 Output:")
22  for animal in animals:
23      print(animal.make_sound())
24  print()

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\AmnonMalka> & C:/Users/AmnonMalka/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/AmnonMalka/AppData/Local/Programs/Python/Python314/Code/Task 4 Abstraction with Base Class.py"
Task 4 Output:
Woof!
Meow!
```

Task 5: Constructor and Destructor

Task 'Introduction and Recap of Object-Oriented Programming (OOP)' Unit 1

```
class Person:
```

```
    def __init__(self, name):
```

```
        # Initialize name
```

```
        self.name = name
```

```
        print(f"{self.name} has been created.")
```

```
    def __del__(self):
```

```
        # Destructor message
```

```
        print(f"{self.name} has been deleted. Goodbye!")
```

Test case

```
print("Task 5 Output:")
```

```
p = Person("Alice")
```

```
del p
```

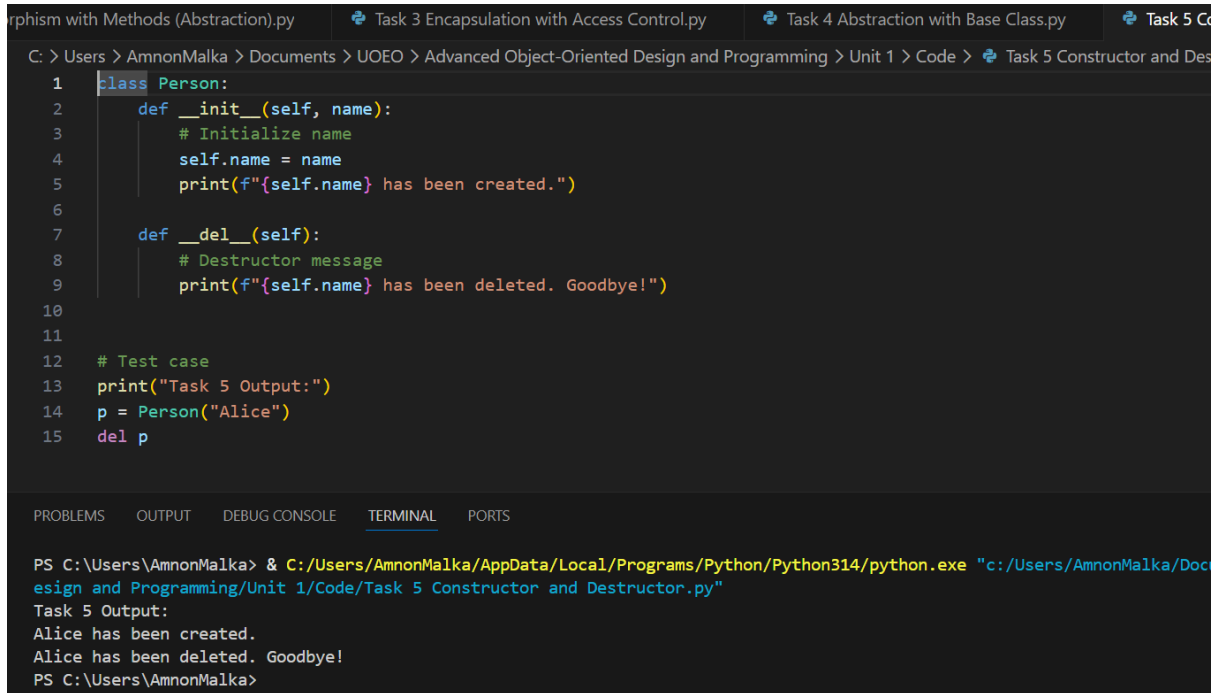
Result:

The results had demonstrated an object lifecycle management in action using constructors and destructors that handle the required initialization and cleanup of objects.

(Downey, 2024).

Task 'Introduction and Recap of Object-Oriented Programming (OOP)' Unit 1

Figure 5: Constructor and Destructor



```
1 class Person:
2     def __init__(self, name):
3         # Initialize name
4         self.name = name
5         print(f"{self.name} has been created.")
6
7     def __del__(self):
8         # Destructor message
9         print(f"{self.name} has been deleted. Goodbye!")
10
11
12 # Test case
13 print("Task 5 Output:")
14 p = Person("Alice")
15 del p
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\AmnonMalka> & C:/Users/AmnonMalka/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/AmnonMalka/Doc
esign and Programming/Unit 1/Code/Task 5 Constructor and Destructor.py"
Task 5 Output:
Alice has been created.
Alice has been deleted. Goodbye!
PS C:\Users\AmnonMalka>
```

Concept Summary

Inheritance promotes code reuse while it also supports clean and structured code, Polymorphism delivers adaptable and extensible interfaces, while it helps improve hygiene, Encapsulation protects data and improves security. It does so by controlling unauthorized access. Abstraction improves simplicity, it enforces a structured approach. These principles are essential for creating scalable, maintainable, and secure systems, particularly in large-scale software engineering, where complexity introduces an extremely negative impact.

(Lott, 2024).

Task 'Introduction and Recap of Object-Oriented Programming (OOP)' Unit 1

References

- Downey, Allen. Think Python. Third edition. Sebastopol, CA: O'Reilly Media, Inc., (2024) Print. Available at: https://essex.primo.exlibrisgroup.com/permalink/44UOES_INST/1b6ojm7/alm-a991008699515607346 [Accessed: 30 April 2026].
- Lott, Steven F. Modern Python Cookbook: 130+ Updated Recipes for Modern Python 3. 12 with New Techniques and Tools. 3rd ed. Birmingham: Packt Publishing, Limited, (2024). Print. Available at: https://essex.primo.exlibrisgroup.com/permalink/44UOES_INST/s7iv0e/cdi_pr-oquest_ebookcentral_EBC31571566 [Accessed: 29 April 2026].

Use of Digital Tools

This document has been produced exclusively for educational purposes. Referenced content, copyrights, and trademarks, remain the sole property of the respective owner. ChatGPT 5.5 was used for some proofreading and clarity as well as literature exploration, language refinement, coding and programming techniques research. All academic writing, analysis, argumentation, and conclusions represent the original work of the author. AI tools were used for exploration, same way as academic literature exploration. No AI-generated content or code was directly copied into the final submission. Any use of content mentioned in this document is properly referenced to its original author and was used responsibly to ensure integrity and originality. All academic writing, analysis, argumentation, and conclusions represent my original

Task ‘Introduction and Recap of Object-Oriented Programming (OOP)’ Unit 1

work. I have used Artificial Intelligence responsibly and ethically, in accordance with the assignment brief. All AI use has been declared and evidenced.